

MISRA C Reflection

Phillip Miavelstuck and Kabir Jolly

While working on Project 2: Railroad Crossing, our group encountered some violations with regard to MISRA Compliance that we addressed.

1. MISRA23_15.7 - Addressed "Else If with no following Else ". MISRA requires tight control flow bounding which we originally did not account for in our implementation. The fix was simple by adding elses where appropriate.
2. MISRA23_10.3 - Addressing the issue of "Value was assigned to an object with a narrower essential type or different essential type category". This warning was fixed by removing redundant casts to void * and by not simultaneously swapping signedness and enlarging integer bit widths (uint32_t -> int64_t).
3. MISRA23_10.8 - Addressing this issue "The value of a composite expression shall not be cast to a different essential type category or a wider essential type" consisted of various steps. First, per our encounters in Project 1, we split most compound expressions up to prevent compound expressions that also changed the type of the resultant (e.g. (float64_t)<int64_t_value> + (float64_t)<int64_t_value>). Other pain points were converting timespec struct values to int64_t instead of time_t. All of these were simple enough fixes after checking with the Understand tool.
4. MISRA23_14.1 - "A loop counter shall not have an essentially floating type". This warning was the result of using the floating point difference of two timespec structs in the unit of seconds to determine how long the loop should last. This was solved by directly comparing the second and nanosecond fields of the two timespecs instead of taking a fractional difference.

On the other hand, there were various MISRA warnings that we intentionally ignored:

1. MISRA23_15.5 - This warning stems from multiple exit points in a function. This warning would be easy to justify suppressing even in professional environments due to it increasing the complexity of code and oftentimes complicating logic where unnecessary.
2. MISRA23_10.3 - Although largely addressed throughout our code, some 10.3 warnings were ignored as casting various values to the desired type was not feasible. Essentially we were in violation due to abiding by the API contract of functions included from standard C library, not from legitimate programming errors.
3. MISRA23_9.5 - "Where designated initializers are used to initialize an array object the size of the array shall be specified explicitly". All instances of this warning were not from array declarations, they were from struct initializations. These were all ignored as each struct was wholly initialized and given values of proper type.

Similar to project 1, writing the multi-threaded code for this project while abiding by

MISRA standards was ~~miserable~~ very informative and allowed us both to grow as software engineers. It also further reinforced our understanding of what is required to develop safety-critical code. Also of note, we had various conditionally compiled blocks of code flagged by Understand as being “unused” or “dead code” despite being used in certain modes of compilation. Overall, most warnings were eliminated and all remaining warnings are minor and do not pose a significant safety risk.